A D PATEL INSTITUTE OF TECHNOLOGY

DEPARTMENT OF INFORMATION TECHNOLOGY

CAMPUSINDEX

STUDENT RECORD SORTING & SEARCHING SYSTEM

A Mini Project Report

A report submitted in partial fulfilment of the requirements for the Degree of

Bachelor of Technology in **Information Technology**

Semester – V

Subject:

Design and Analysis of Algorithm (202045601)

By

Jenil Patel (12402080601053)

Faculty Guide

Minakshi Patil

Assistant Professor

Academic Year 2026–27

Department of Information Technology

A D Patel Institute of Technology

New Vallabh Vidyanagar, Vitthal Udyognagar – 388121

Anand (Gujarat), India

CERTIFICATE

This is to certify that the mini project work embodied in this report entitled “CampusIndex: Student Record Sorting & Searching System” was carried out by **Jenil Patel (12402080601053)** at A D Patel Institute of Technology for partial fulfilment of the requirements of the Bachelor of Technology degree awarded by Charutar Vidya Mandal (CVM) University. This project work has been carried out under the guidance and supervision of the faculty guide and is to the satisfaction of the department.

Date: _____

Place: New Vallabh Vidyanagar

Lab Teacher
Minakshi Patil
Assistant Professor

Head of Department
Dr. Priyang Bhatt
Associate Professor & Head

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to A D Patel Institute of Technology and the Department of Information Technology for providing the opportunity and resources to undertake this mini project, “CampusIndex: Student Record Sorting & Searching System.”

I am thankful to my faculty guide and lab teacher for their guidance, suggestions, and support throughout the development of this project. Their feedback helped me understand the design, implementation, and testing of the system.

I also thank my classmates and everyone who provided assistance or encouragement during the project. Finally, I am grateful to my family for their continuous support.

Jenil Patel
(12402080601053)

TABLE OF CONTENTS

Certificate — i
Acknowledgement — ii
List of Tables — iv
Abstract — v
CHAPTER 1: INTRODUCTION — 7
1.1 Background
1.2 Problem Statement
1.3 Objectives
1.4 Scope
1.5 Existing and Proposed System
1.6 Applications
CHAPTER 2: THEORETICAL BACKGROUND — 10
2.1 Searching and Sorting
2.2 Linear Search
2.3 Binary Search
2.4 Sorting Algorithms
2.5 Complexity Analysis
CHAPTER 3: SYSTEM DESIGN AND METHODOLOGY — 13
3.1 System Overview
3.2 System Modules
3.3 Working of the System
3.4 Algorithm Steps
3.5 Complexity Analysis
CHAPTER 4: IMPLEMENTATION AND TESTING — 15
4.1 Technologies Used
4.2 Implementation Details
4.3 Test Cases and Results
4.4 Limitations
CHAPTER 5: CONCLUSION AND FUTURE SCOPE — 18
References

LIST OF TABLES

Table 1. Technologies used in the project

Table 2. Functional requirements

Table 3. Comparison of searching algorithms

Table 4. Comparison of sorting algorithms

Table 5. Test cases for record management and algorithms

ABSTRACT

Student information needs to be organized, searched, and sorted efficiently. CampusIndex is a browser-based mini project that combines a student record management interface with an interactive Design and Analysis of Algorithms learning environment. It allows users to add, edit, delete, filter, and sort student records containing a name, student ID, department, academic year, CGPA, and marks.

The project demonstrates Linear Search and Binary Search for locating a student by ID. It also visualizes six sorting algorithms—Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, Quick Sort, and Heap Sort—on fields such as marks, CGPA, name, and student ID. A benchmark module compares algorithms using comparison counts, writes, elapsed time, and theoretical complexity information.

The application is implemented with HTML, CSS, and dependency-free JavaScript. Student records are stored in the browser using localStorage, so the data remains within the current browser profile rather than being uploaded to a server. The interface includes an overview dashboard, student records, sorting studio, search studio, benchmark lab, complexity guide, and CSV export.

The main educational goal is to connect algorithmic theory with a practical interface. The project illustrates why sorting can improve the suitability of a search method, how algorithm growth rates differ, and how empirical timings may vary with the device and browser. It is intended as a learning demonstration rather than a centralized production student information system.

CHAPTER 1: INTRODUCTION

1.1 Background

Educational institutions maintain records containing student identities and academic information. These records often need to be searched by student ID or name and sorted by marks, CGPA, department, or other fields. As a dataset grows, the choice of algorithm can significantly affect the amount of work required to find or organize information.

Design and Analysis of Algorithms studies how to solve computational problems correctly and efficiently. Searching algorithms locate a target item, while sorting algorithms arrange data in a defined order. CampusIndex provides a practical environment in which these techniques can be applied to student records and their behaviour observed.

1.2 Problem Statement

The problem is to develop a simple, interactive web application that manages student records and demonstrates the operation and performance characteristics of classic searching and sorting algorithms. The system should allow records to be maintained and filtered while helping users compare algorithms using theoretical complexity and practical measurements.

1.3 Objectives

• Create an accessible student record management interface.
• Add, edit, delete, filter, and sort student records.
• Demonstrate Linear Search and Binary Search using student IDs.
• Visualize six sorting algorithms on selected record fields.
• Compare performance through comparisons, writes, and elapsed time.
• Present time complexity, auxiliary space, and sorting stability.
• Export the dataset as a CSV file.

1.4 Scope

The project focuses on small-scale student record management and algorithm education in a modern web browser. It includes a sample dataset, dashboard statistics, record management controls, search and sorting visualizations, a benchmark view, a complexity reference, and CSV export. The project can be opened directly from the deployed GitHub Pages site or run locally by opening index.html in a modern browser.

Records are stored using the browser's localStorage. Data is associated with the current browser profile and is not automatically synchronized between devices or users. The project does not claim to provide centralized storage, authentication, or production-grade institutional record security.

1.5 Existing and Proposed System

In a manual workflow, a user may scan records individually, reorder data by hand, or rely on a spreadsheet without seeing how the underlying algorithms work. These approaches may be adequate for a small list but provide limited insight into algorithmic cost.

The proposed CampusIndex system combines record management with visual demonstrations. Users can manage records in one view and explore sorting, searching, benchmarks, and complexity in dedicated modules.

1.6 Applications

• Demonstration of searching and sorting algorithms in DAA labs.
• Practice with student record datasets and field-based ordering.
• Comparing complexity with empirical counters and timings.
• Learning browser-based JavaScript and local data persistence.
• Exporting datasets for spreadsheet software.

1.7 Functional Requirements

The principal functional requirements are summarized below. They describe features visible in the project interface and repository documentation.

ID	Requirement	Expected behaviour
FR-01	Manage records	Add, edit, and delete student records.
FR-02	Search and filter	Filter records by name, ID, department, and year where supported.
FR-03	Sort records	Order records by name, ID, CGPA, marks, or department.
FR-04	Visualize algorithms	Run sorting animations and search traces.
FR-05	Benchmark	Compare counters and elapsed time for available sorting algorithms.
FR-06	Export data	Download the current records in CSV format.

1.8 Non-functional Requirements

The application should be responsive, understandable, and usable in a modern browser without a separate installation. Algorithm demonstrations should provide clear feedback. Since data is stored locally, users should understand that clearing browser data or changing browser profiles can affect access to saved records.

CHAPTER 2: THEORETICAL BACKGROUND

2.1 Searching and Sorting

Searching is the process of finding an item that matches a target key. Sorting arranges a collection in a defined order according to a chosen key. In this project, a student is represented as a record containing multiple fields, and an algorithm compares the selected field when processing the collection.

2.2 Linear Search

Linear Search checks items sequentially from the beginning until the target is found or all records have been checked. It does not require the collection to be sorted. Its best-case time complexity is O(1), while average and worst-case time complexities are O(n). Auxiliary space is O(1) for an iterative implementation.

2.3 Binary Search

Binary Search operates on an ordered collection. It compares the target with the middle item and discards the half that cannot contain the target. Its best-case time is O(1), and average and worst-case time are O(log n). The input must be sorted according to the same key used for the search.

In CampusIndex, the Search Studio traces Linear Search and Binary Search for a selected student ID. The project documentation notes that student IDs are sorted before Binary Search, and the displayed comparison count refers to the search phase.

2.4 Sorting Algorithms

Bubble Sort repeatedly compares adjacent elements and swaps them when they are out of order. Selection Sort repeatedly selects the smallest remaining element and places it in the next position. Insertion Sort grows a sorted prefix by inserting each new element into its correct location.

Merge Sort divides the dataset into smaller parts, sorts those parts, and merges them. It has O(n log n) time complexity in best, average, and worst cases, with O(n) auxiliary space. Quick Sort partitions items around a pivot; its average time is O(n log n), while its worst case is $O(n^2)$. Heap Sort uses a heap to repeatedly select the next element and runs in O(n log n) time in best, average, and worst cases.

Table 3. Complexity summary

Algorithm	Best	Average	Worst	Auxiliary space	Stable	True
Bubble Sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes	
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)	No	
Insertion Sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes	
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)	Yes	
Quick Sort	O(n log n)	O(n log n)	$O(n^2)$	O(log n) avg.	No	
Heap Sort	O(n log n)	O(n log n)	O(n log n)	O(1)	No	

2.5 Algorithm Analysis and Big-O Notation

Big-O notation describes an upper bound on how an algorithm's work grows with input size. O(1) represents constant work, O(log n) logarithmic growth, O(n) linear growth, O(n log n) efficient comparison-sorting growth, and $O(n^2)$ quadratic growth. When the number of records doubles, an $O(n^2)$ algorithm can require roughly four times as much work for large inputs, while an O(n log n) algorithm usually scales more efficiently.

2.6 Time and Space Complexity

Time complexity estimates the growth in operations as input size increases. Space complexity estimates the additional memory required. A sorting algorithm may use fewer comparisons but require additional memory, as Merge Sort does, or may sort in place with little auxiliary space, as Heap Sort typically does. Stability is also relevant: a stable sort preserves the relative order of records with equal keys.

2.7 Benchmark Interpretation

The Benchmark Lab compares the available sorting methods on the same dataset and displays comparison counts, writes, elapsed time, and complexity. Comparison and write counts are often more repeatable than elapsed time, which varies with browser workload, processor performance, and dataset size. The benchmark supports learning rather than providing a universal ranking of algorithms.

Theoretical complexity describes growth patterns; empirical results describe a particular run under particular conditions. Both views should be interpreted together.

CHAPTER 3: SYSTEM DESIGN AND METHODOLOGY

3.1 System Overview

CampusIndex is a client-side web application. Its user interface is defined in HTML, styled with CSS, and made interactive through JavaScript. It runs in a browser and stores the current student dataset in localStorage. The main navigation exposes six views: Overview, Student Records, Sorting Studio, Search Studio, Benchmark Lab, and Complexity Guide.

3.2 System Modules

Overview Dashboard: Displays total student count, average CGPA, top performer, and search-complexity information. It also provides a recent-record view and shortcuts to algorithm modules.

Student Records: Supports record creation, editing, deletion, filtering, sorting, and CSV export. Fields include name, student ID, department, year, CGPA, and marks.

Sorting Studio: Visualizes Bubble, Selection, Insertion, Merge, Quick, and Heap Sort. Users can select the sort field and observe algorithm steps and operation metrics.

Search Studio: Traces Linear Search and Binary Search for a selected student ID and compares the search process.

Benchmark Lab: Compares all six sorting algorithms on the same input using operation counts and elapsed time.

Complexity Guide: Summarizes best, average, and worst time complexity, auxiliary space, and stability.

3.3 Working of the System

1. Open the deployed CampusIndex page in a modern browser.
2. Review the overview dashboard and sample records.
3. Add, edit, or delete records; changes are saved in browser localStorage.
4. Use record search to filter by name, ID, department, or year.
5. Choose a sort field to arrange the records.
6. Open Sorting Studio, select one of six algorithms and a record field, then start the visualization.
7. Open Search Studio, select a target student ID, and run Linear Search and Binary Search.
8. Run Benchmark Lab to compare sorting algorithms on the same data.
9. Review Complexity Guide to relate the results to Big-O analysis.
10. Export the dataset to CSV when a portable copy is required.

3.4 Algorithm Steps: Linear Search

1. Start from the first record.
2. Compare the current record’s ID with the target ID.
3. If they match, return the record and stop.
4. Otherwise move to the next record.
5. If the end is reached, report that the target was not found.

3.5 Algorithm Steps: Binary Search

1. Arrange student IDs in sorted order.
2. Set low and high boundaries for the search range.
3. Calculate the middle position and compare its ID with the target.
4. If the IDs match, return the record.
5. If the target is smaller, continue in the left half; otherwise continue in the right half.
6. Repeat until the target is found or the range becomes empty.

3.6 Complexity Analysis

	Time complexity	Condition / explanation	True
n	Best O(1); average/worst O(n)	Checks records sequentially; works on unsorted data.	
n	Best O(1); average/worst O(log n)	Requires sorted IDs; count refers to search phase.	
	Average/worst $O(n^2)$	Adjacent comparisons and swaps.	
	O(n log n)	Uses additional memory to merge sorted parts.	
	Average O(n log n); worst $O(n^2)$	Performance depends on partition behaviour.	
	O(n log n)	Uses heap operations and constant auxiliary space.	

For n student records, search and sorting costs depend on the selected algorithm. Binary Search is asymptotically faster than Linear Search for large ordered collections, but the data must be sorted first. Sorting methods with O(n log n) growth generally scale better than quadratic-time methods for larger datasets.

CHAPTER 4: IMPLEMENTATION AND TESTING

4.1 Technologies Used

	Purpose	True
	Defines the structure, views, tables, forms, dialogs, and controls.	
	Styles the responsive dashboard, navigation, tables, and visualizations.	
	Implements record management, algorithms, visualization, metrics, and CSV export.	
	Persists student data within the current browser profile.	
	Hosts the static web application.	
	Can be used for editing source files and running/testing the application.	

4.2 Implementation Details

The project repository contains index.html, styles.css, app.js, and a README file. The HTML file defines application views and dialogs, the CSS file provides visual styling, and JavaScript handles the dataset, navigation, algorithms, visualization, and browser storage. A sample dataset is provided so that the interface can be explored immediately.

The application uses a localStorage key for student data. The current data is not uploaded to a server, and the project does not require a database installation or build step to open the static page in a modern browser.

4.3 Test Cases and Results

The following test cases provide a practical checklist for validating the submitted version. Record the actual status after manually running each case; the table does not claim unverified tests have passed.

Test ID	Input / action	Expected result	Status	True
TC-01	Add student	Record appears and dashboard values update.	To verify	
TC-02	Edit a record	Updated values display and save locally.	To verify	
TC-03	Delete a record	Record is removed from the dataset.	To verify	
TC-04	Search known ID	Matching record is found.	To verify	
TC-05	Search missing ID	No-match result is shown.	To verify	
TC-06	Run sorting visualizations	Records are ordered by selected key.	To verify	
TC-07	Run benchmark	Counters, timing, and complexity display.	To verify	
TC-08	Export CSV	CSV containing current records downloads.	To verify	

4.4 Limitations

• Data is local to the current browser profile and is not a centralized database.
• Login, role-based access, and multi-user synchronization are not included in the documented scope.
• Timing measurements vary by device, browser, and dataset size.
• Binary Search requires the searched IDs to be ordered.
• The application is an educational demonstration rather than a secure institutional information system.

CHAPTER 5: CONCLUSION AND FUTURE SCOPE

5.1 Conclusion

CampusIndex brings together student record management and core concepts from Design and Analysis of Algorithms. The application provides record creation and editing, filtering, sorting, CSV export, algorithm visualizations, benchmark comparisons, and a complexity guide. Linear Search and Binary Search illustrate how the structure of the input affects search performance, while six sorting algorithms demonstrate different time-space trade-offs.

The project uses HTML, CSS, and JavaScript and stores records in the browser with localStorage. Its interactive interface makes theoretical concepts easier to explore. The work also highlights the difference between theoretical complexity and measured runtime, which can vary with the environment. CampusIndex is suitable as a DAA mini project and can be extended with stronger persistence, testing, and security.

5.2 Future Scope

• Add secure authentication and role-based access.
• Integrate a server-side database with backup and recovery.
• Add CSV import, input validation, and duplicate checks.
• Provide downloadable benchmark reports and repeatable test settings.
• Add automated tests for search, sorting, record operations, and export.
• Improve accessibility and step-by-step explanations for each algorithm.

REFERENCES

1. CampusIndex — Student Record Sorting & Searching System, deployed project: https://jenilpatel0112.github.io/DAA-Project-Algorithm-Binary-Search-Student-Record-Sorting/

2. Project repository README and source files: index.html, styles.css, app.js, and README.md.

3. Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. E., *Introduction to Algorithms*, MIT Press.

4. Sedgewick, R. and Wayne, K., *Algorithms*, Addison-Wesley.

5. MDN Web Docs, JavaScript and Web Storage API documentation.

6. GitHub Pages documentation for static website hosting.